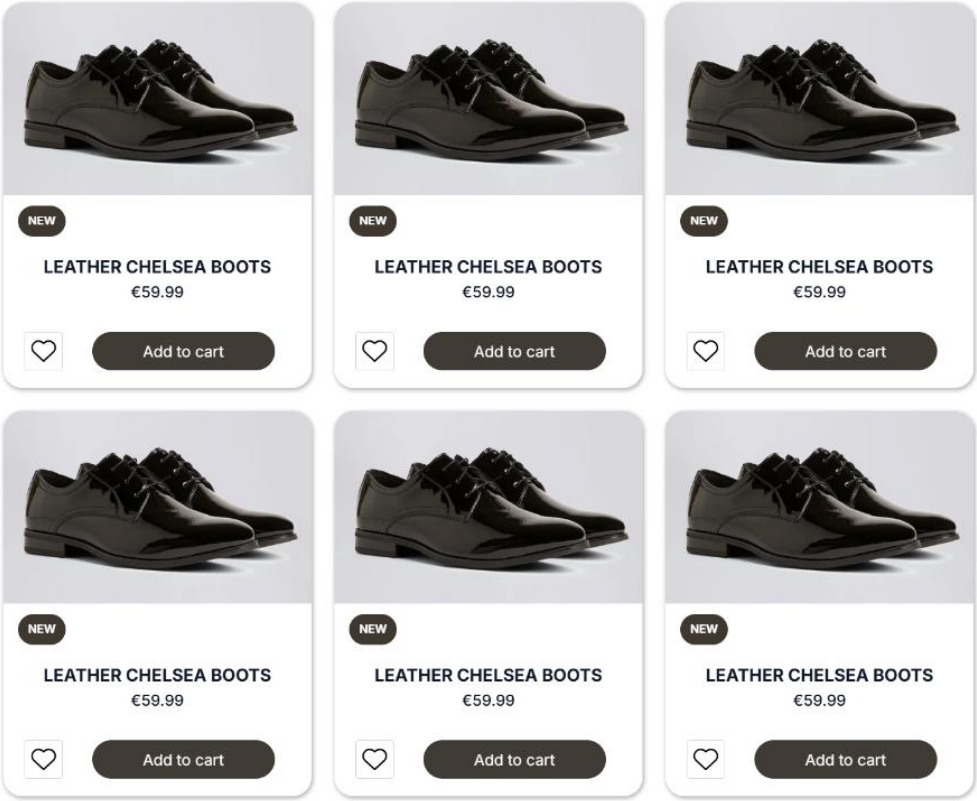
Testing UX/UI component builders/designers:

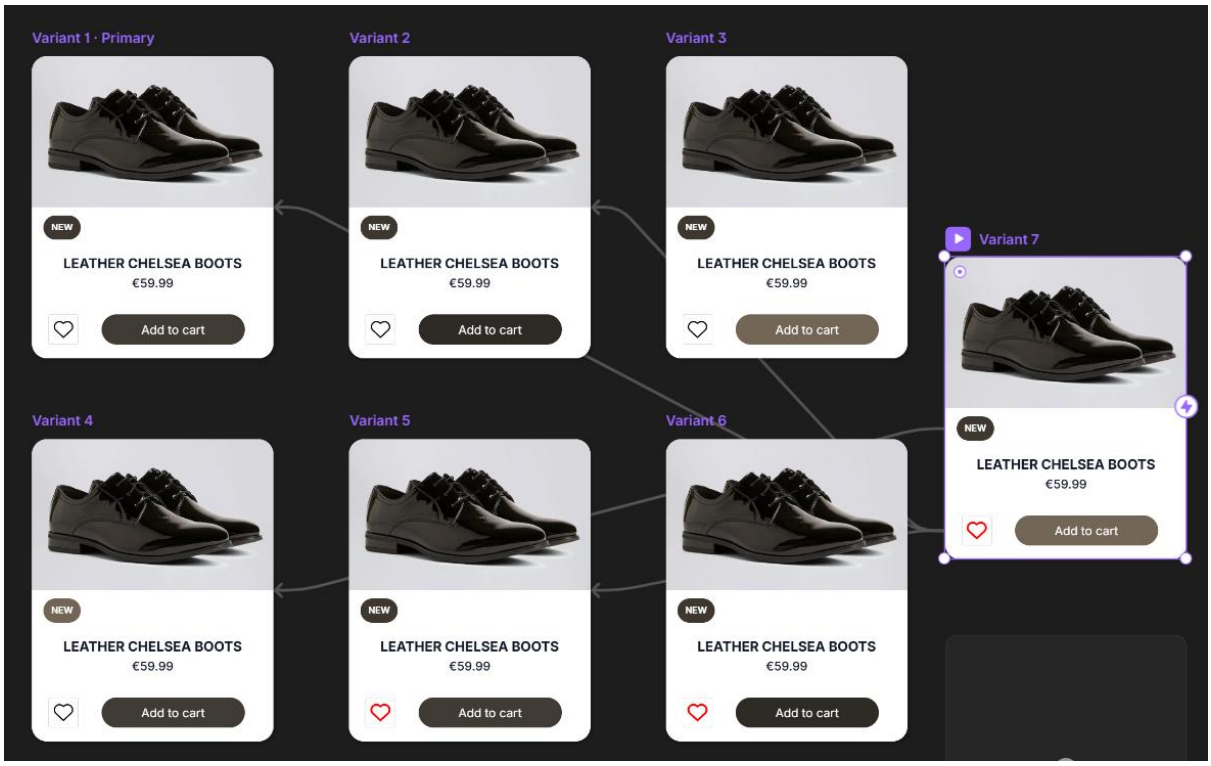
Ease of Use: How intuitive is the UI? Can you build quickly?

- **Component Reusability**: Can you export or reuse components in React?
- **Tailwind/Catalyst Compatibility**: Does it support Tailwind CSS or integrate with Catalyst UI Kit?
- **Export Options**: Can you export to code (HTML/CSS/React)?
- **Collaboration & Versioning**: Does it support team workflows or version control?
- **Shopify API Integration Potential**: Can you design components that map to Shopify product data?

Framer:

- **Card Design**:
  - Built with image, title, price, button, heart icon, and "NEW" badge.
- **Grid Layout**:
  - Used Auto Layout with wrap and spacing.
  - Added multiple card instances.
- **Interactions**:
  - Created variant for button shadow for click and hover.
  - Created New tag hover variant.
  - Created Favourite Click variant
- **Findings**:
  - Variants allow smooth, no-code interactions.
  - Layout is responsive and preview is available – for main desktop page.


NEW
LEATHER CHELSEA BOOTS
€59.99
Add to cart


Variant 1 · Primary
Variant 2
Variant 3
Variant 4
Variant 5
Variant 6
Variant 7
NEW
LEATHER CHELSEA BOOTS
€59.99
Add to cart

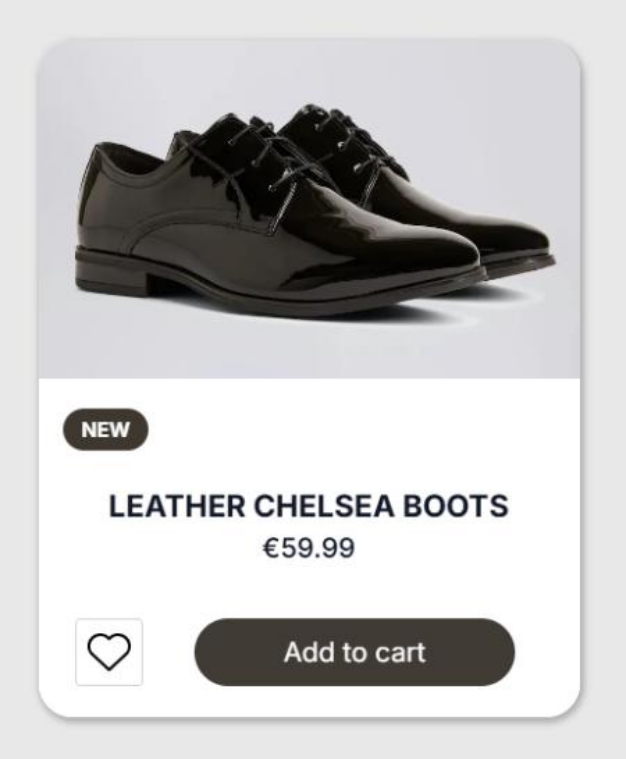


Figure 1 - Framer Card

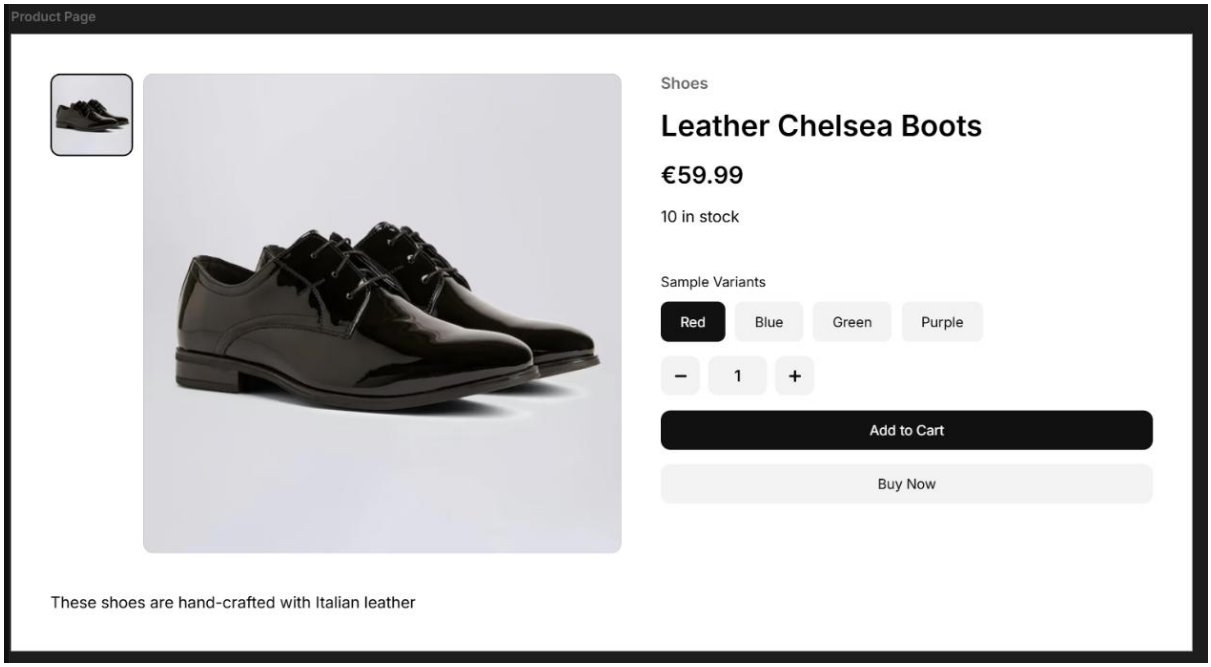


Figure 2 - Frameship product component

Feature	Summary	Research Source
Ease of Use	Framer offers a Figma-like interface with drag-and-drop layout, auto layout, and real-time preview.	TechRadar Review
Component Reusability	You can create reusable components with props and variants. For React export, use the React Export plugin – but subscription is required for usage, or unframer CLI to convert Framer components into typed React code – GitHub Repo.	React Export Plugin
Tailwind/Catalyst Compatibility	Framer doesn't natively support Tailwind or Catalyst, but you can manually style components using Tailwind classes in code components. Tailwind + Framer Motion (animation library for react, JavaScript, Vue) integration is possible.	Catalyst Docs
Export Options	Framer doesn't support direct HTML/CSS/JS export. React export is available via plugins. General code creation from components is available to copy.	Pentaclay Export Guide, Framer Marketplace Plugin
Collaboration & Versioning	Framer supports real-time collaboration, permissions, and version history. You can view published versions, revert changes, and share staging links.	Framer Help – Staging & Versions
Shopify API Integration Potential	Framer supports Shopify integration via plugins like **Frameship**, allowing you to sync product data and build e-commerce UIs.	Frameship Plugin, SegmentUI Guide

Negatives:

Limited code exportation

Plugin only for exports

Rebuilding components multiple times for usage outside of framer

Doesn't support Tailwind CSS and Catalyst UI

Styling manually applied in component code

Plugins are restricted mostly – subscriptions

Proto.io:

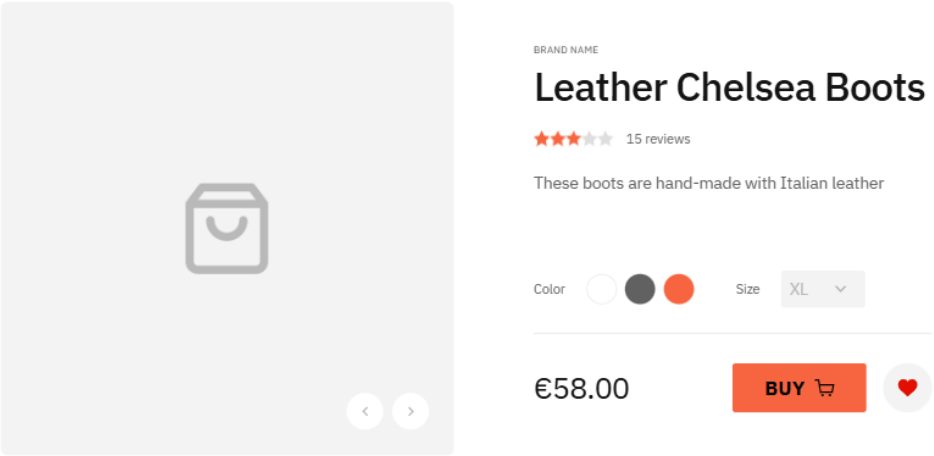


Figure 3 - Premade Component

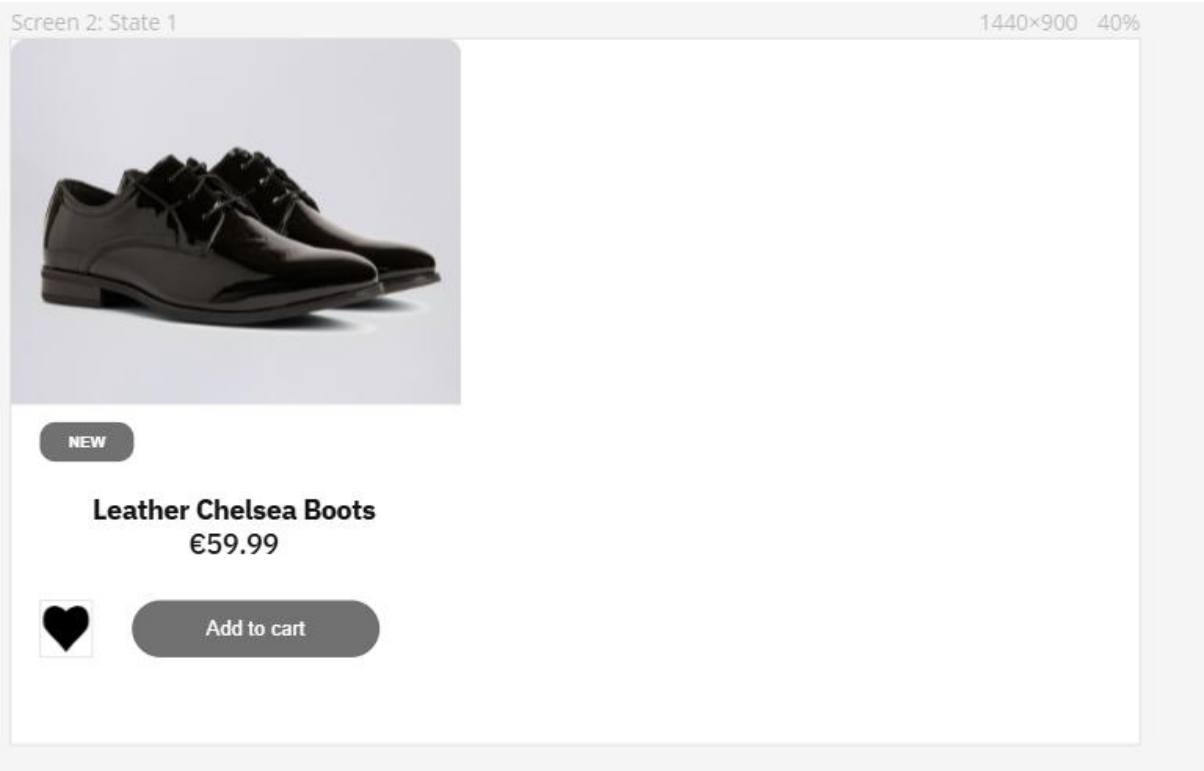


Figure 4 - Card Created - Slower

Feature	Summary	Research Source
Ease of Use	Proto.io offers a drag-and-drop interface with pre-built UI elements and templates. It's beginner-friendly and fast for building layouts/prototypes. Manual Grids available to create with ruler tool.	Proto.io Overview
Component Reusability	Components can be grouped and reused across screens, but there's no direct export to React or modular code. Reusability is visual only.	
Tailwind/Catalyst Compatibility	Proto.io does not support Tailwind CSS or Catalyst UI integration. Styling is done visually or via custom CSS, but not with utility-first frameworks.	
Export Options	You can export prototypes as HTML. There's no React or Tailwind export.	
Collaboration & Versioning	Supports team collaboration with shared projects and version history. You can preview and test interactions in-browser.	Proto.io Features

Feature	Summary	Research Source
Shopify API Integration Potential	Proto.io is not designed for backend or API integration. You can simulate product cards visually.	Proto.io Limitations

Negatives:

No code exportation apart from HTML

Components are not exportable, only visually reusable

No API integrations available

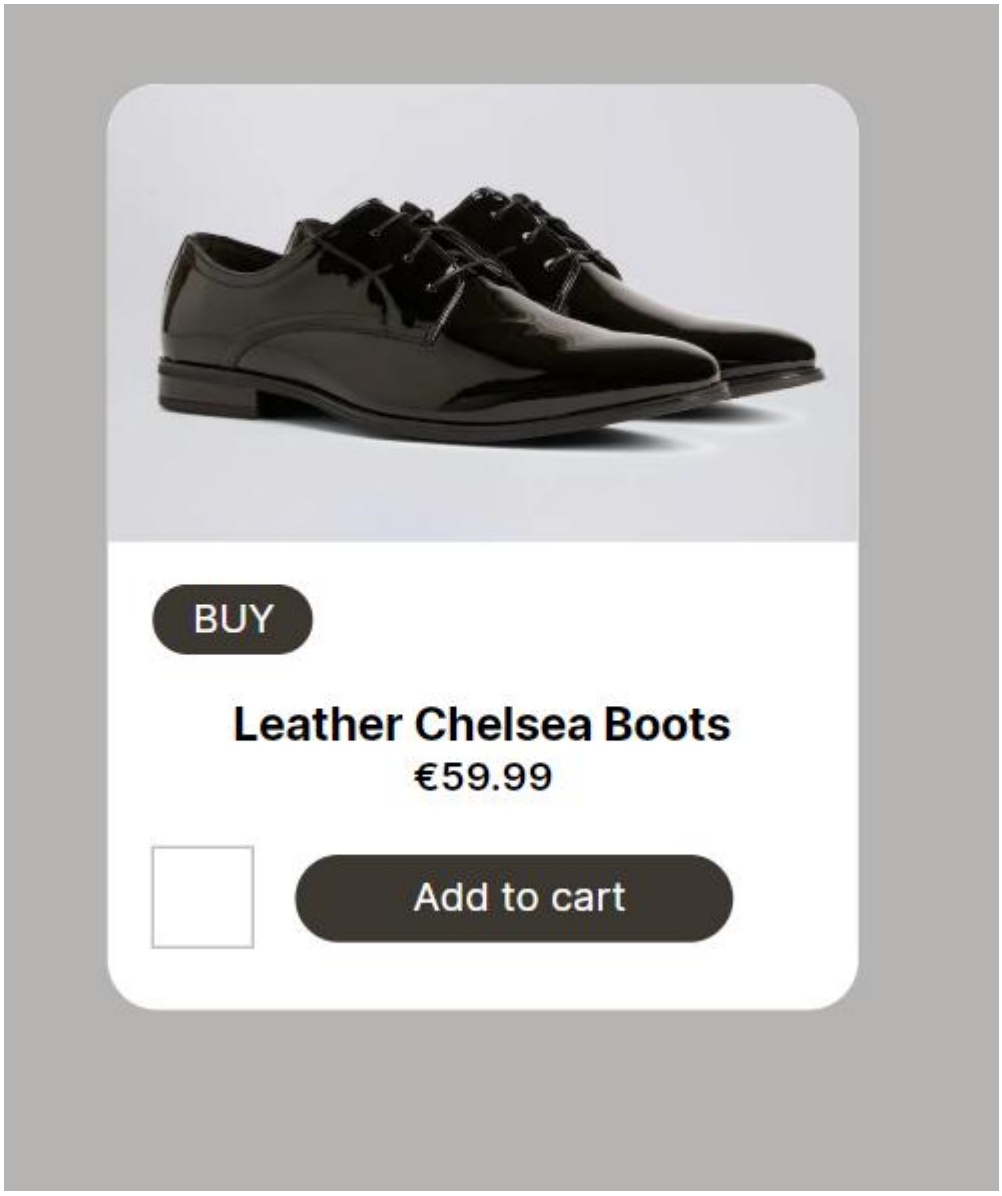
Tools for creating are manual and time-restricting

Lunacy:

Errors with lunacy while creating state instances of components – forces reload each time.

Auto layout for frame grids is effective but labelled poorly.

CSS exports available in browser. HTML link export



Feature	Summary	Research Source
Ease of Use	Lunacy offers a clean, Figma-like interface with drag-and-drop layout and built-in assets. It's fast, lightweight, and works offline. Built-in Grid builder component.	BNR OK Tutorial
Component Reusability	You can create reusable components with multiple states. Editing the main component updates all instances.	Lunacy Docs – Components

Feature	Summary	Research Source
Tailwind/Catalyst Compatibility	Lunacy does not natively support Tailwind or Catalyst.	Lunacy Docs
Export Options	You can export designs as PNG, SVG, or HTML. No direct React or Tailwind code export.	GitHub – Lunacy Docs
Collaboration & Versioning	Supports real-time collaboration, comments, and cloud sync.	
Shopify API Integration Potential	Lunacy does not support backend or API integration.	Lunacy Docs – Limitations

Negatives:

No code exports

No API integrations

Only visual styling and HTML reusability

Uizard:

No component function – only duplication of grouped items.

Can make instances to change screen or move to part of screen, but none for hoverables or click designs.

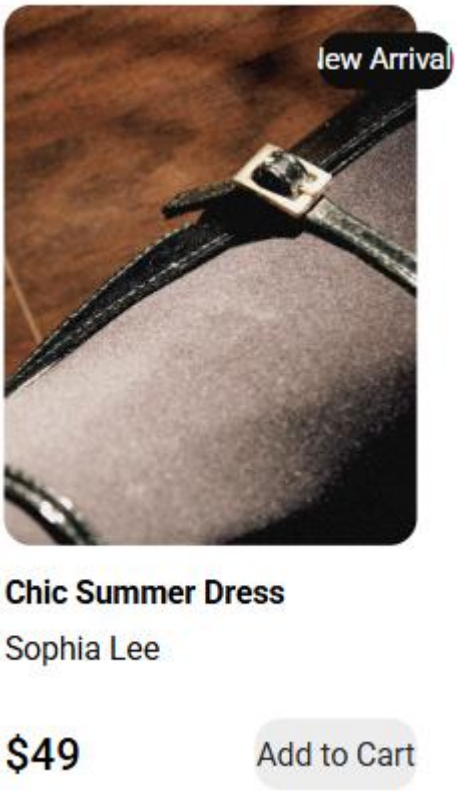


Figure 5 - Uizard Pre-made Template – Card

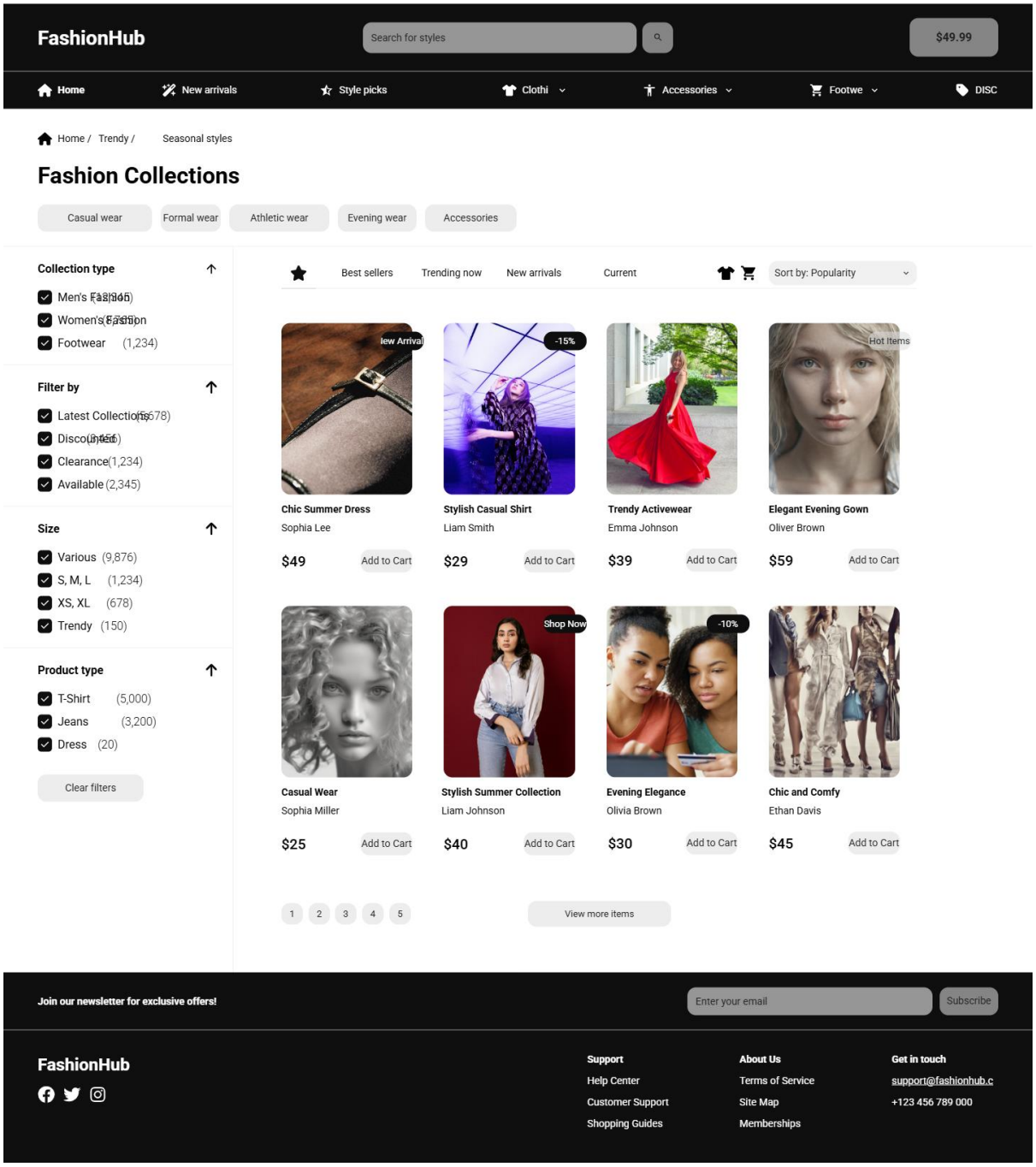


Figure 6 - AI generated template

Feature	Summary	Research Source
Ease of Use	Uizard offers a drag-and-drop interface with AI features like Screenshot-to-Design, Text-to-UI, and Magic Theme. Wireframe scanner available and AI theme generation.	Uizard for Developers

Feature	Summary	Research Source
Component Reusability	Components can be reused visually and styled with themes.	Using UI Components in Uizard – YouTube
Tailwind/Catalyst Compatibility	Uizard does not support Tailwind CSS or Catalyst UI integration.	Uizard Docs
Export Options	You can export designs as images, PDFs, or HTML previews. There is support for React and CSS code export – subscription required.	Uizard AI Overview
Collaboration & Versioning	Supports real-time collaboration, comments, and version control with project history.	
Shopify API Integration Potential	Uizard does not support backend or API integration.	

Negatives:

Code export requires subscription and only for React and CSS

No API integration

Focuses on fast layouts/wireframes/prototypes through AI but often visual differences between content created.

Penpot:

Detailed grid maker with justify options

Components are easy to edit and create

Auto Layout editing is clear and simple

Previews available

CSS overview of grids and layout available in browser

CSS code available

HTML and SVG code available for copying

Grid fidelity is loss when css and html copied over to code.

NEW

Leather Chelsea Boots

€59.99

Add to cart

NEW

Leather Chelsea Boots

€59.99

Add to cart

NEW

Leather Chelsea Boots

€59.99

Add to cart

NEW

Leather Chelsea Boots

€59.99

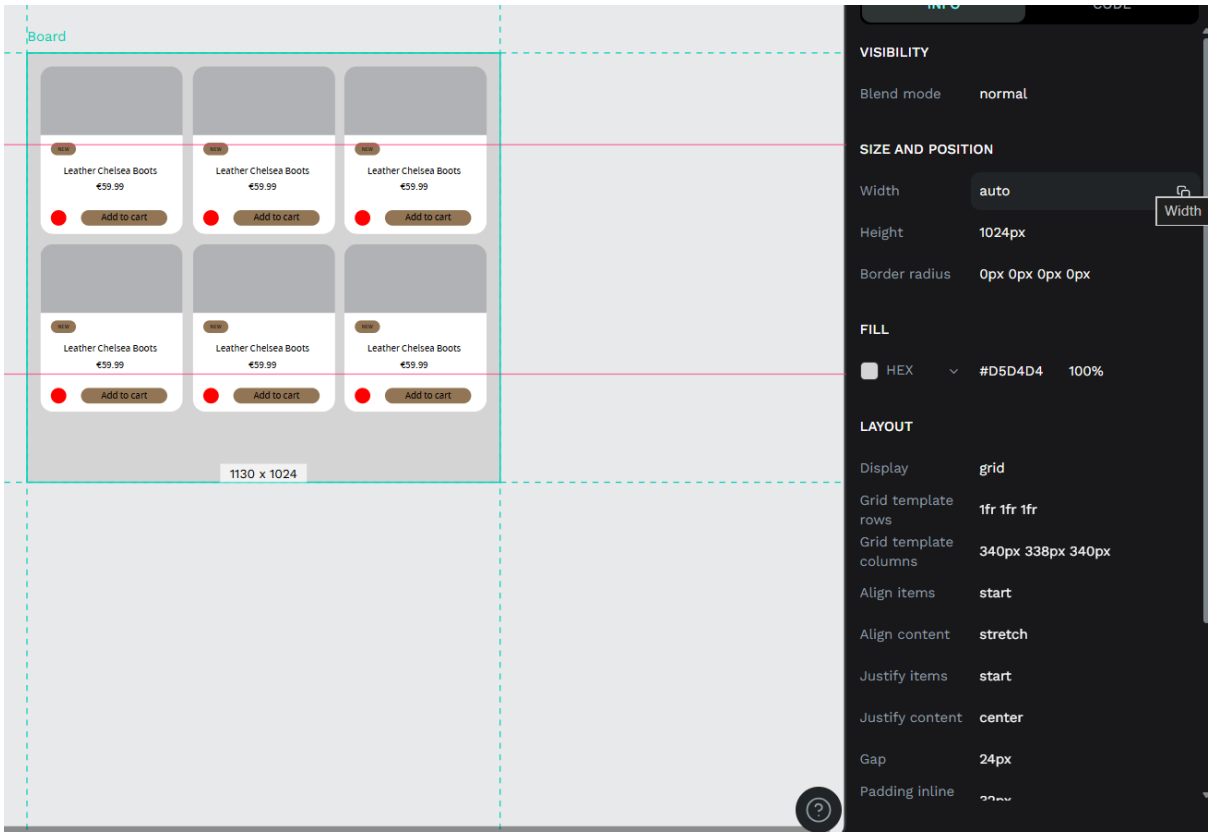
Add to cart

NEW

Leather Chelsea Boots

€59.99

Add to cart



Research Area	Why It Matters
Component Reusability	Can you build modular, scalable UI components like product cards and reuse them across screens?
Design System Support	Does Penpot support tokens, shared libraries, and consistent styling across components?
Code Export & Handoff	Can developers easily inspect and extract code (CSS, SVG)? Is it production-ready or just for reference?
Tailwind/Catalyst Compatibility	Can you manually apply Tailwind-style classes or export styles that map to utility-first frameworks?
API/Data Integration Potential	Can Penpot simulate or bind to real data (e.g. Shopify)? If not, how does that limit your workflow?
Collaboration & Versioning	How well does Penpot support team workflows, comments, and version control compared to Framer or Figma?

Build.layoutit:

Easy to use

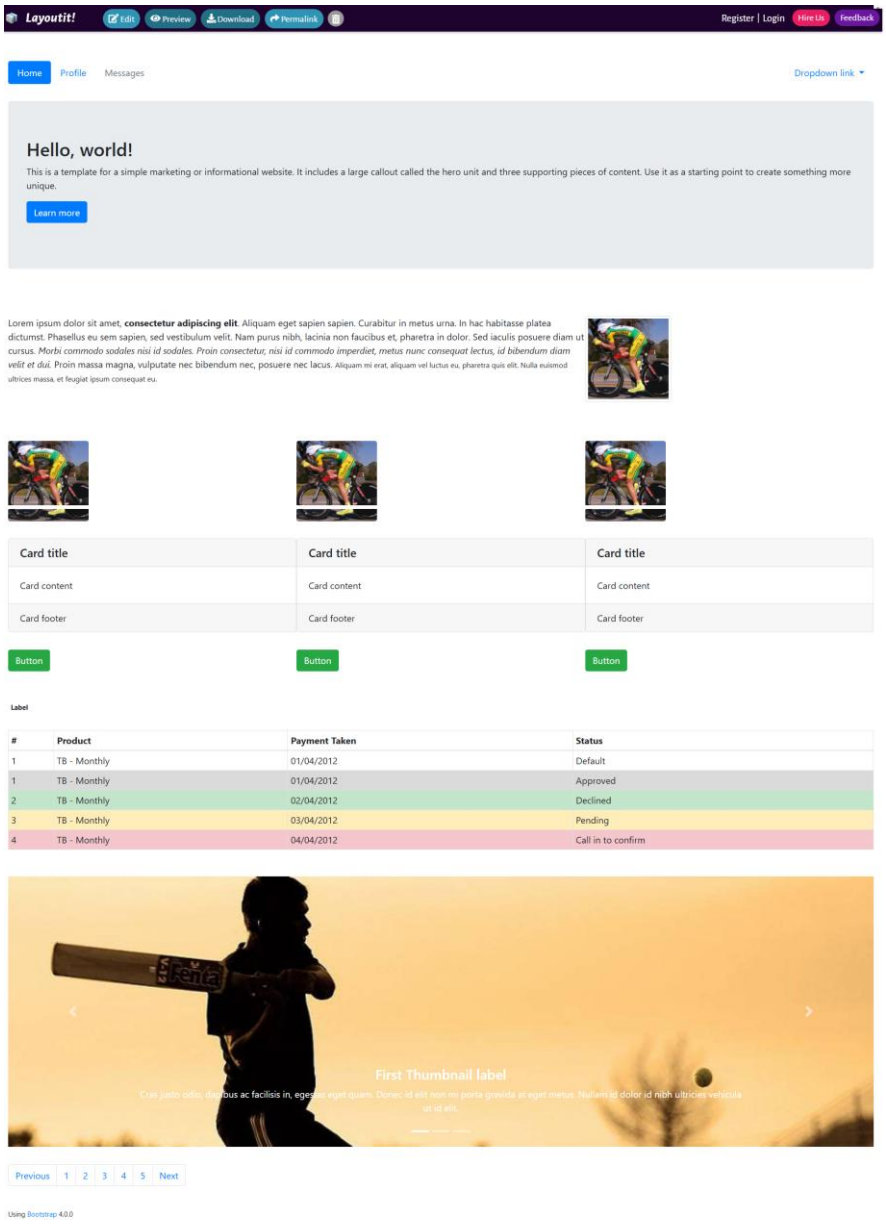
Prototype layout builder

Hard to edit pre-made content

HTML code available to download as zip or copy in browser

No CSS available to copy/download

Lacking styling options – very bare webpage designs and limited



Grid.layoutit:

Grid builder with box layouts and labelling

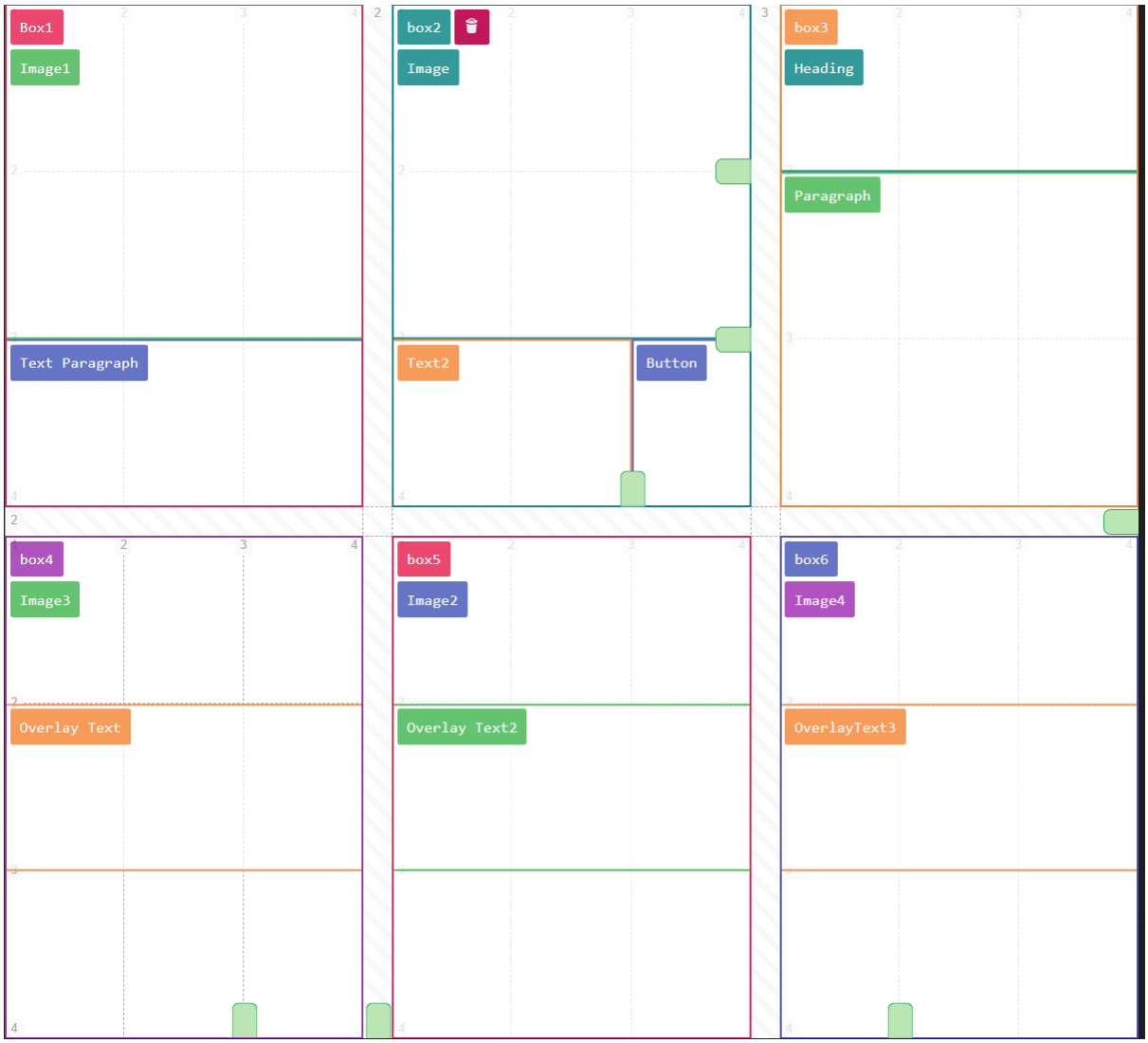
Code available for HTML

Code available for CSS

In browser editing of code available for members

Editors available – Sandbox, CodePen, Stackblitz

Clear simple layout but effective base planning for design scheme.



Cssgridgenerator.io:

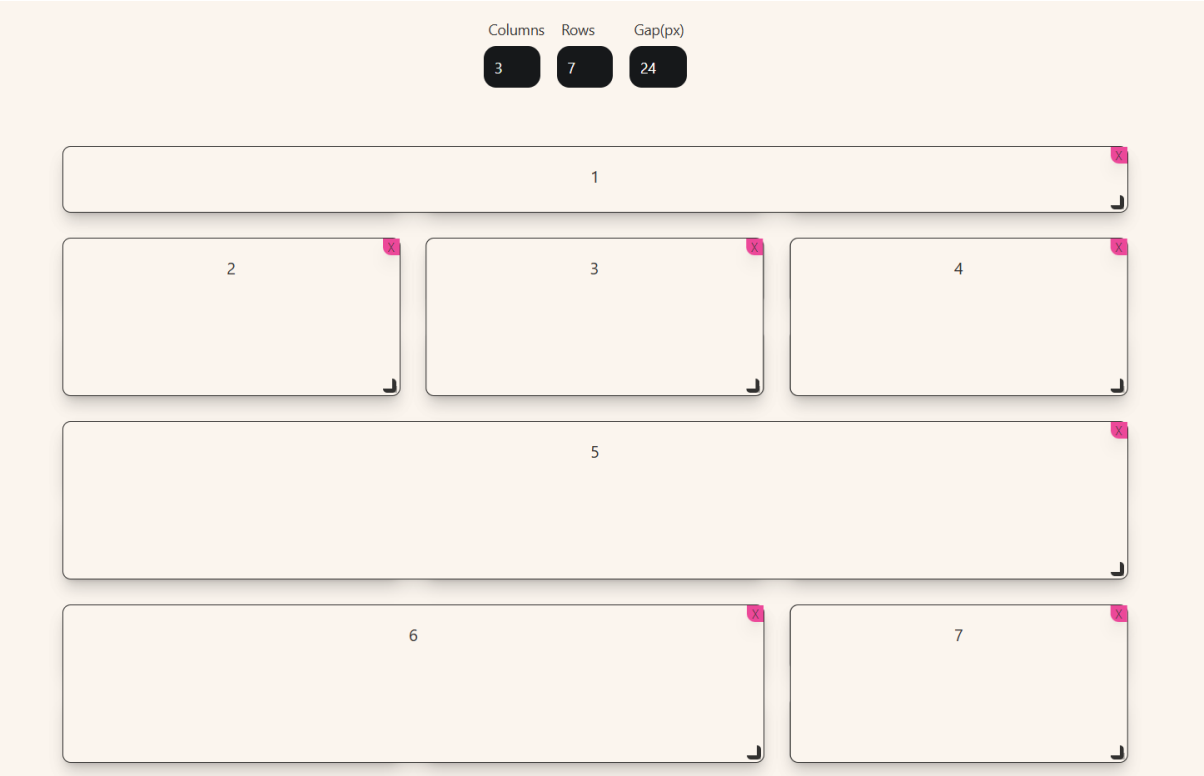
Clear design, easy to follow instructions

No coding experience required

Simple but effective results

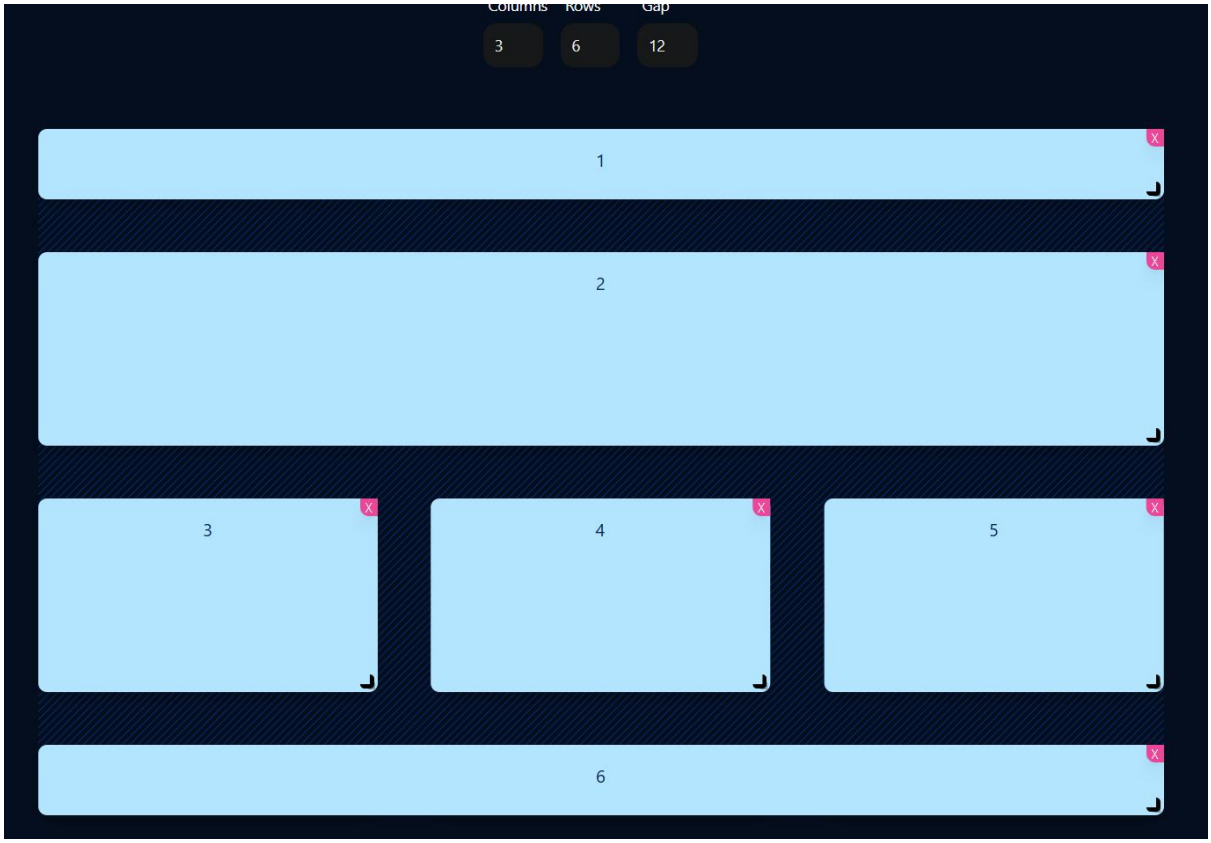
In-browser code for HTML and CSS

Responsive design real-time



Neighbouring Website for Tailwind:

Tailwindgen.com



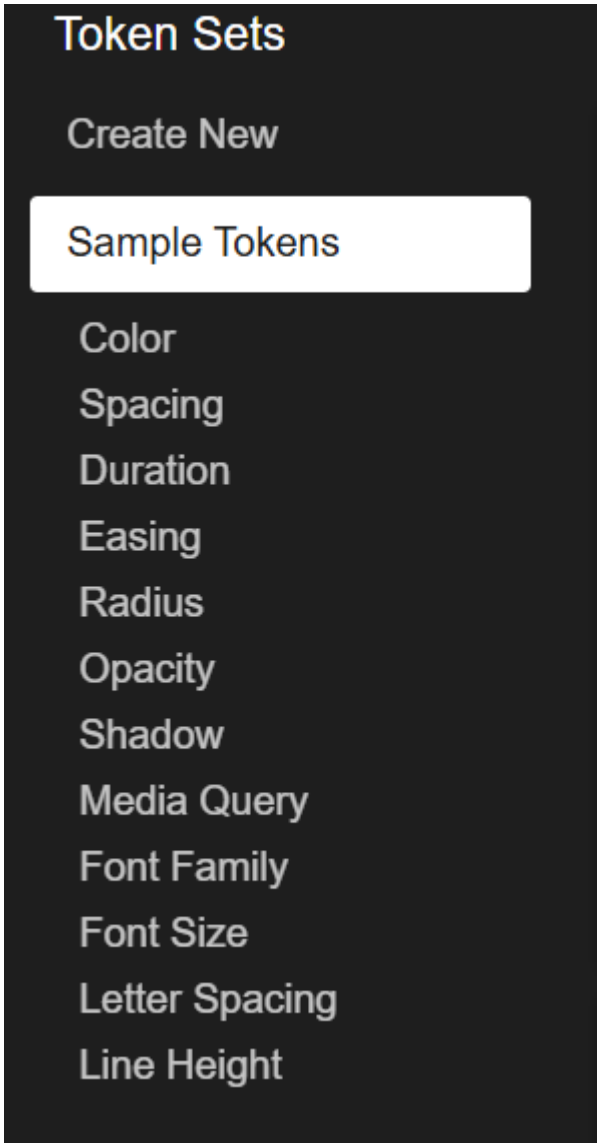
Same system for designing grids

Returns JSX or HTML code with Tailwind inline code

Tokens.layoutit:

Provides multiple ranges of tokens for styling

Colour, spacing, text size, opacity, font family, letter spacing, line height



Returns code in W3C, CSS, SASS

Fields include name, value, description

Easily trackable, creatable and usable

ShopifyAPI:

Overview:

Shared Strengths Across Most Platforms

- **Ease of Use**: Most tools offer drag-and-drop interfaces with auto layout or grid builders (Framer, Proto.io, Lunacy, Uizard, Penpot).
- **Visual Prototyping**: All platforms support visual UI creation, often with pre-built components or templates.
- **Basic Export Options**: HTML/CSS export is available in most tools (Framer, Penpot, Uizard, Layoutit, CSSGridGenerator).
- **Component Reusability (Visual)**: Most platforms allow visual reuse of components across screens or layouts (Framer, Proto.io, Lunacy, Uizard, Penpot).

Commonly Missing or Limited Features

Feature	Missing or Limited In
Tailwind/Catalyst Compatibility	Not natively supported in any platform; only Framer allows manual Tailwind styling in code components
React Export	Only Framer and Uizard support React export, and both require subscriptions or plugins
Shopify API Integration	Only Framer supports Shopify integration via plugins (Frameship); others lack backend/API support
Production-Ready Code Export	Most platforms export HTML/CSS for reference, not clean
Tailwind Tokens	No platform offers native support for Tailwind tokens
Code Fidelity	Exported code often loses layout fidelity or lacks styling (Penpot, Layoutit, CSSGridGenerator)

Final Summary: What's Missing Across the Board

- **Unified Tailwind support**: No platform offers native integration with utility-first frameworks or design systems.
- **Modular React + Tailwind export**: Only Framer and Uizard offer partial support, and both are gated by subscriptions.

- **Shopify-ready components**: Framer is the only platform with Shopify plugin support; others lack API integration.

Penpot, framer, Uizard, proto.io, lunacy, build.layoutit, cssgridgenerator, tokens.layoutit, grid.layoutit,

RESEARCH PART 2:

KIGEN UI:

Colour

- Provides colour palettes
- Light / Dark mode palettes
- Tailwind Library

Typography

- Font styles
- Text Type

Misc

- Scaling values for scaling variables (e.g. sm, md, lg)

Present

- Overview of all active variables of the plugin

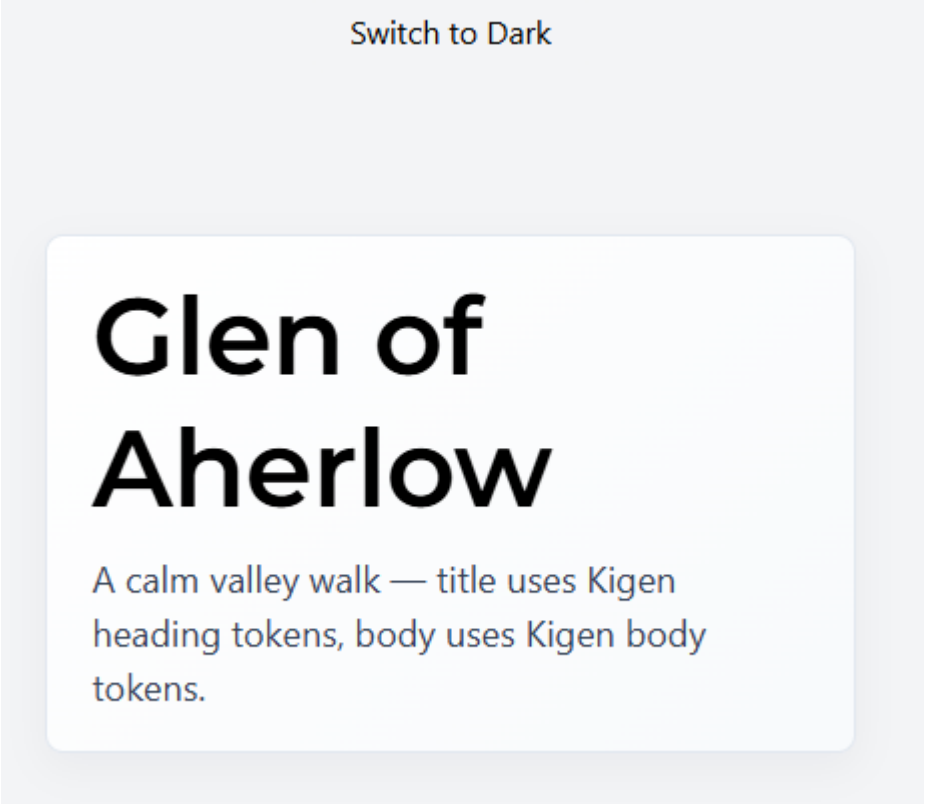
Export

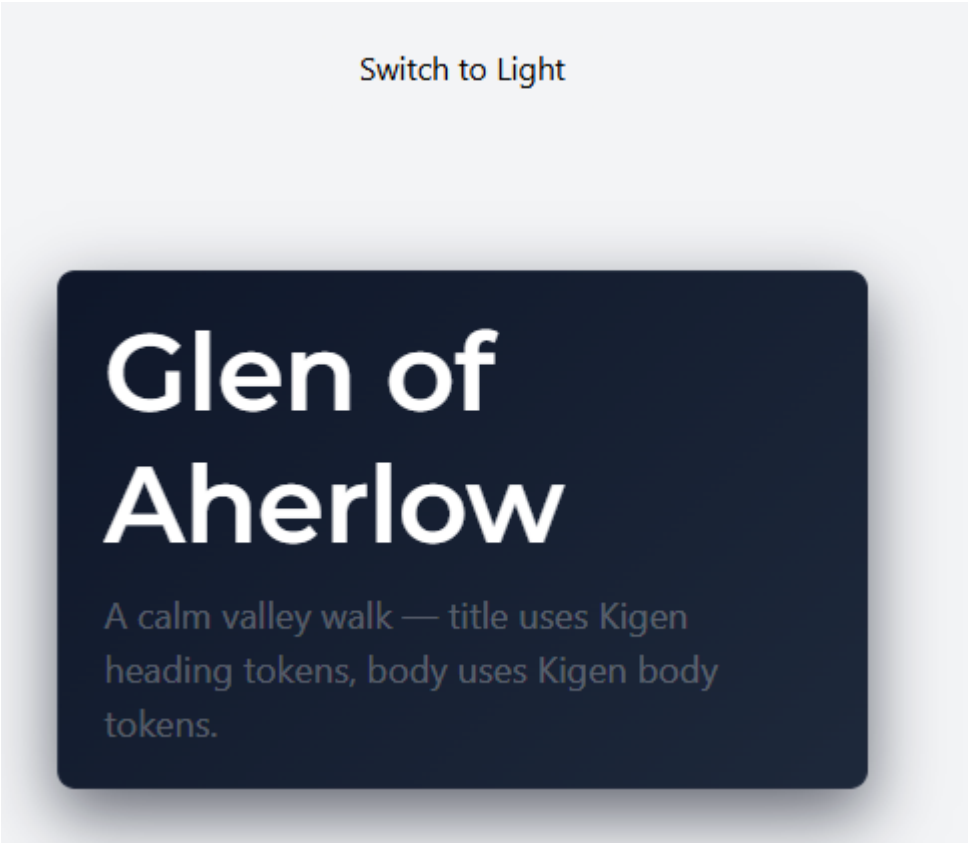
- Can export any of the active variables
- Only exportable as CSS, JSON, JS (WIP)

Notes:

- Colour doesn't work for instant updating
- Update : colour tokens update when variable details are set to tailwind and 50,100,200...etc then re-generated
- Can use colour tokens for text strokes...etc
- Can add typography attached as variables – can detach to edit further
- Misc – Radius scaling works – can still be edited for separate corners
- Exported downloads keep same file name

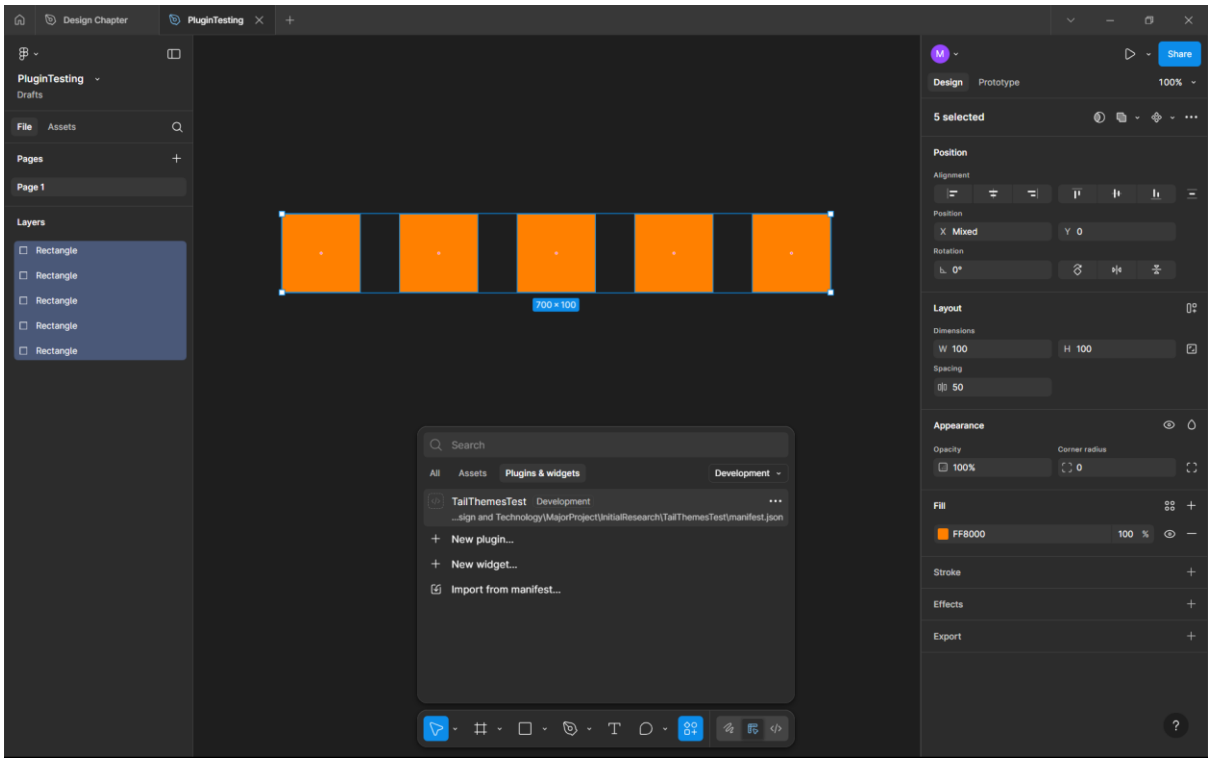
-

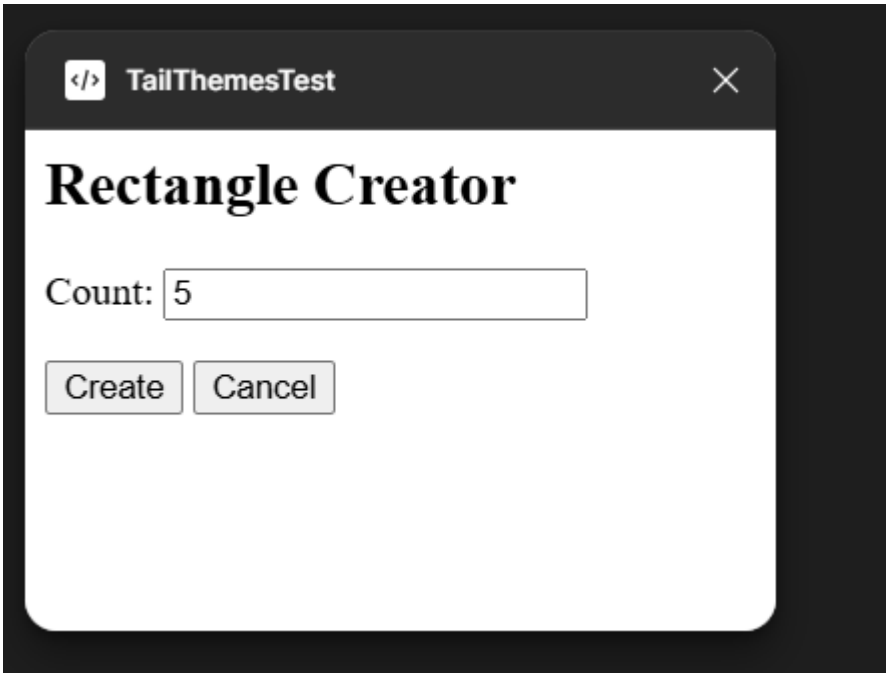
- Card created using exported information
- Able to create new token designs
- 

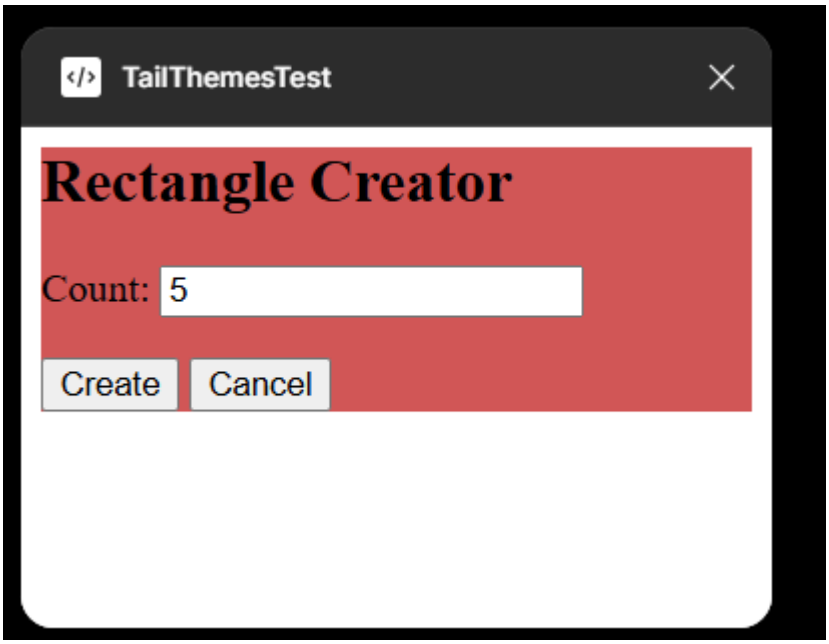
- 

- Light and dark modes available for all colour tokens palette

Plugin Creation:



- Created base plugin with premade code
- https://developers.figma.com/docs/plugins/plugin-quickstart-guide/
- Uses typescript, node with npm commands and eslint for error checking
- Uses terminal task build npm watch to update ts to js
- 

- Plugin Design basic
- 

- Ui.html page added div with css colour style
- https://help.figma.com/hc/en-us/articles/4407275338775-BYFP-Introduction-to-Plugins-API
- https://developers.figma.com/docs/plugins/
- Good guides ^^

- Main thread runs in sandbox - minimal js environment
- Heirarchy – DocumentNode -> PageNode – 1 docNode per figma, multiple pageNodes.
- PageNodes consist of layers for the figma page while the docNode is the entire folder
- Plugins work on current pageNode so each page stores independently
- Figma has colour tokens

  `--figma-color-text-brand` Blue or purple text used for things like links.

  `--figma-color-text-danger` Red text used to alert users about an error.

  `--figma-color-text-warning` Yellow text used to warn users about a potential problem.

  `--figma-color-text-success` Green text used for confirmation, and approval.

  Background Colors

  `--figma-color-bg` Our default white / light grey background fill for most views (like our file browser, editor sidebars, panels, windows, and modals).

  `--figma-color-bg-secondary` A light grey fill used to create sections on top of a background, as well as for some inputs.

  Common Hues

  `--figma-color-bg-brand` Blue or purple fill colors for backgrounds like a blue primary button.

  `--figma-color-bg-danger` Red fill color behind destructive buttons, and error banners.

  `--figma-color-bg-warning` Yellow fill color for warning banners

  `--figma-color-bg-success` Green fill color for confirmation banners.

  Borders

  `--figma-color-border` Our default border color for dividers between sections.

  `--figma-color-border-strong` The darker border color we use for secondary outline buttons.

  `--figma-color-border-selected` The blue or purple border color we use for focused / selected inputs.
-
- Light/dark modes available

- Dev mode is used to create code generations that figma doesn't support usually.
- Available to use different languages and frontend frameworks through dev mode.
- Edit manifest.json to allow devMode

- manifest.json for Dev Mode

```json
{
  "name": "Plugin for inspection",
  "id": "000000000000000000",
  "api": "1.0.0",
  "main": "code.js",
  "documentAccess": "dynamic-page",
  "editorType": ["dev"],
  "capabilities": ["inspect", "vscode"]
}
```

- Basic manifest.json for plugins for code generation

```json
{
  "name": "MyCodegenPlugin",
  "id": "{your id here}",
  "api": "1.0.0",
  "main": "code.js",
  "editorType": ["dev"],
  "capabilities": ["codegen", "vscode"],
  "codegenLanguages": [
    {"label": "React", "value": "react"},
  ]
}
```

-
- Example ^^
- Codegens:
- Reads user's current selection – sends figma relevant code in layer – code is rendered in inspect panel

Plugin manifest with multiple languages

Example manifest.json including all codegenPreference itemTypes and multiple languages

```
1  {
2    "name": "MyPlugin",
3    "id": "{your id here}",
4    "api": "1.0.0",
5    "main": "code.js",
6    "editorType": ["dev"],
7    "capabilities": ["codegen"],
8    "codegenLanguages": [
9      {"label": "React", "value": "react"},
10     {"label": "Typescript", "value": "typescript"}
11   ],
12   "codegenPreferences": [
13     {
14       "itemType": "unit",
15       "scaledUnit": "Rem",
16       "defaultScaleFactor": 16,
17       "default": true,
18       "includedLanguages": ["react"]
19     },
20     {
21       "itemType": "select",
22         "propertyName": "tabSize",
23         "label": "Tab Size",
24         "options": [
25           {"label": "2", "value": "2", "isDefault": true },
26           {"label": "4", "value": "4"},
27           {"label": "8", "value": "8"}
28         ],
29         "includedLanguages": ["typescript"]
30     },
31     {
32         "itemType": "action",
33         "propertyName": "showMore",
34         "label": "More settings...",
35         "includedLanguages": ["typescript"]
36     }
37   ]
38 }
```

-
- Example of all codegen preferences, itemTypes and languages

Simple example

```
figma.codegen.on("generate", () => {
  return [
    {
      language: "HTML",
      code:
        `<div class="container">
          <div class="box"></div>
        </div>`,
      title: "HTML",
    },
    {
      language: "CSS",
      code:
        `.container {
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
          }
          .box {
            width: 100px;
            height: 100px;
            background-color: #000;
          }
        `,
      title: "CSS",
    },
  ];
});
```

-

HTML

```html
<div class="container">
  <div class="box"></div>
</div>
```

CSS

```css
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
.box {
  width: 100px;
  height: 100px;
  background-color: #000;
}
```

-
- Example of css and html codegen code versus result in inspect dev mode
- https://help.figma.com/hc/en-us/articles/4407531247639-BYFP-Building-Your-Plugin#h_01HRB1KF5Y0RSXPYEQ9VKNNQ4M
- For further developing basics ^^

https://v3.tailwindcss.com/docs/plugins

for adding tailwind plugins to third party plugins – including utility themes

https://decipher.dev/30-seconds-of-typescript/docs/hexToRGB/

https://medium.com/@dianagureev/hex-rgb-converter-on-javascript-e6f4daadce5f

for creating HEX to RGB

Existing plugins :

1. Developer Profile & Roles

Findings:

- Respondents included Frontend Developers, Full-Stack Developers, UI Engineers, UX/UI Designers, and Students.
- Experience ranged from 0–5+ years.
- Team sizes varied: solo developers to teams of 10+.
- UI involvement ranged from low to moderate; some respondents had full ownership, others mainly implemented designs.

Outcome / Implications:

- Target audience spans both junior and mid-level developers, often juggling multiple roles.
- Solutions need to cater to developers who implement rather than design.
- Automation benefits should address both solo developers and team workflows.

2. Frontend Workflow & Development Process

Findings:

- Typical workflows: from low-fidelity wireframes - high-fidelity designs - coding - deployment.
- Coding often begins after final designs, but some start during design iterations or with minimal input.
- Design changes are common, especially during iteration, which can slow implementation.
- Validation methods include visual comparison, pixel-perfect checks, and review meetings.

Outcome / Implications:

- There is frequent manual interpretation of designs, causing delays and potential inconsistencies.
- A design-to-code tool can reduce manual checking and layout misinterpretation, especially during frequent design changes.

3. Design Tools & Prototyping

Findings:

- Most use Figma, with some using Adobe XD, Sketch, or Penpot.
- Designs are primarily used for layout, spacing, component structure, and interactions.
- Prototypes/wireframes often do not fully reflect real implementation constraints.
- Missing elements often include full functionality, detailed layout measurements, and responsive behavior.

Outcome / Implications:

- There is a gap between design tools and actual code, especially regarding responsiveness and spacing.
- Automation could bridge this gap by translating design specs directly into maintainable code.

4. Figma Deep Dive

Findings:

- Features relied on: Components, Constraints, Inspect Panel.
- Confusion arises from: Auto layout rules, Nested layouts, Spacing, Responsive sizing, Typography.
- Inspect panel values are inconsistent across respondents.

Outcome / Implications:

- Developers spend time interpreting Figma outputs, which is error-prone.
- Project could standardize Figma-to-code output, reducing guesswork and inconsistencies.

5. Tailwind CSS & Styling Practices

Findings:

- Tailwind CSS is used for rapid prototyping and maintaining consistency.
- Challenges: Non-standard spacing, utility class complexity, overcomplicated code in larger projects.

- Solutions include converting to closest Tailwind scale, arbitrary values, modifying design, or custom configs.
- Benefits: Faster prototyping, consistency, smaller CSS files, easier maintenance.

Outcome / Implications:

- Developers manually map design spacing - Tailwind classes, which is time-consuming and error-prone.
- The automation tool could handle class conversion, especially for non-standard or inconsistent spacing, improving workflow speed.

6. Component Building & React

Findings:

- React is the most common framework; others use Vue or Angular.
- Component structures vary: Feature-based, Page-based, Atomic design, or flexible approaches.
- Refactoring happens frequently due to design changes.
- Time-consuming aspects: Layout, Spacing, Responsiveness, State handling, Styling.

Outcome / Implications:

- Developers struggle with repetitive component adjustments due to design changes.
- Automated code generation could pre-fill layouts and spacing logic, reducing manual refactoring effort.

7. Code Generation & Automation

Findings:

- Mixed experience: Some used code generation regularly; others never.
- Challenges: Poor code quality, lack of flexibility, framework mismatch, missing component structure.
- Desired features: Readable, framework-compatible, utility-first CSS compatible, editable, accurate structure, minimal repetition.

Outcome / Implications:

- Automation tools are relevant but underused due to quality and flexibility issues.
- The project could focus on producing clean, editable, framework-compatible code, directly addressing these pain points.

8. Plugin Usage & Expectations

Findings:

- Plugin usage is inconsistent; some never use them.
- Current limitations: Poor quality, inaccurate translation, limited framework support, performance issues.
- Desired features: Accurate layout conversion, editable clean code, responsive support, utility-first outputs.

Outcome / Implications:

- There is a clear gap in reliable plugins for design-to-code workflows.
- The project could fill this gap by providing a plugin that translates Figma layouts into reusable, clean, framework-compatible code.

9. Project Validation & Relevance

Findings:

- Manual conversion of inline CSS/design - Tailwind classes is common.
- Developers see value in automation for speed, consistency, and reducing errors.
- Most useful workflow stages: Design handoff, component development, refactoring.

Outcome / Implications:

- Automation can save hours per project, reduce inconsistencies, and improve maintainability.
- The tool has potential team-wide benefits, particularly for upgrading legacy or inconsistent codebases.

10. Final Reflections

Findings:

- Biggest inefficiencies: Organizing layouts, client uncertainty, interpreting designs.
- Desired tooling improvements: Visualization tools, more reliable design-to-code workflows, better spacing/layout automation.

Outcome / Implications:

- The project could reduce workload, minimize errors, and speed up workflow across design-to-code stages.

Overall Conclusion

1. Pain Points Identified:
    - Manual interpretation of design specs
    - Inconsistent Figma outputs (spacing, layout, inspect panel)
    - Time-consuming Tailwind class mapping
    - Frequent component refactoring due to design changes
    - Poor quality/limitations of current code generation plugins
2. Opportunities for Your Project:
    - Automate Figma - code conversion (Tailwind, React)
    - Maintain consistent layout, spacing, and component structure
    - Produce editable, readable code compatible with frameworks
    - Reduce manual verification and refactoring time
    - Provide team-wide standardization benefits
3. Relevance & Outcome:
    - Strong support for the necessity of design-to-code automation, across student and professional developers.
    - The tool is relevant for saving time, improving accuracy, and scaling development workflows.
    - Developers not heavily involved in design (full-stack or backend-focused) benefit from consistent code generation and class mapping.